

MailForge 邮件服务器

SMTP + POP3 + Web + 两层端到端加密
课程设计汇报

苏俊玮、王诗贺

计算机与网络课程设计·选题 18

2026 年 9 月 11 日

原生 Socket · 手写协议状态机 · 自研 AES/ChaCha20 · WebCrypto–OpenSSL 互通

一、原理篇

1. 项目背景与需求映射
2. 总体架构与运行模型
3. SMTP / POP3 协议原理
4. Web 后端与 REST 接口
5. 两层加密的密码学原理
6. AES-256-CBC / ChaCha20 实现

二、结构篇

1. 代码目录与模块划分
2. 类关系与关键数据结构
3. 邮件落盘、会话与密钥管理
4. 构建、运行与测试入口

三、创新与改进

1. 六个创新点
2. 测试结果与演示脚本
3. 可继续修改的方向
4. 总结与问答准备

项目简介: MailForge 是什么

- **从零实现**: 不依赖现成 SMTP/POP3/HTTP 协议库, 使用原生 Socket 手写文本协议状态机。
- **完整闭环**: 浏览器 Web 界面 → HTTP 后端 → SMTP/POP3 客户端 → 邮件服务器 → .eml 落盘。
- **多用户隔离**: 每个账号独立目录 mailbox/<user>/, 支持注册、登录、收发、附件、删除、已发送。
- **两层加密**:
 1. Web ↔ 服务器: AES-256-GCM + RSA-OAEP-2048 数字信封;
 2. 服务器内部 / 落盘: 纯 C++ 自研 AES-256-CBC 或 ChaCha20。
- **可演示、可压测**: 浏览器内手敲 SMTP/POP3 命令; 一键连发 100 封约 1MB 邮件并统计时延、成功率、解密还原率。

技术关键词

C++17 / POSIX Socket / 多线程 /
RFC 5321 / RFC 1939 / RFC 5322 /
MIME / OpenSSL EVP / WebCrypto
/ AES-256 / ChaCha20 / RSA-2048
/ PKCS#7 / Base64

运行端口

SMTP: 2525
POP3: 1110
HTTP: 8080

课程需求与实现映射

课程要求	MailForge 实现
SMTP 标准交互	EHLO / MAIL FROM / RCPT TO / DATA / QUIT, DATA 点填充与解析
POP3 标准交互	USER / PASS / STAT / LIST / RETR / DELE / RSET / QUIT, 三阶段状态机
邮件正文与附件 $\leq 2.1\text{MB}$	MIME multipart + Base64 附件; 发送侧大小保护; 压测附件约 1MB
传输过程机密性 不少于 2 种可选算法	两层加密: Web RSA 信封 + 服务器端口间/落盘 AES/ChaCha AES-256-CBC、ChaCha20 可切换; Web 层另有 AES-GCM + RSA-OAEP
平均时延 $< 2\text{s}$ 成功率 $\geq 99\%$	压测 100 封约 1MB 邮件, AES 约 100ms、ChaCha 约 60ms 级 内建 100 封连续压测, 统计 SMTP 成功、收件、丢包率、解密还原率
前端操作流畅	Web 邮箱: 登录、写邮件、收件箱、阅读、附件、已发送、压测

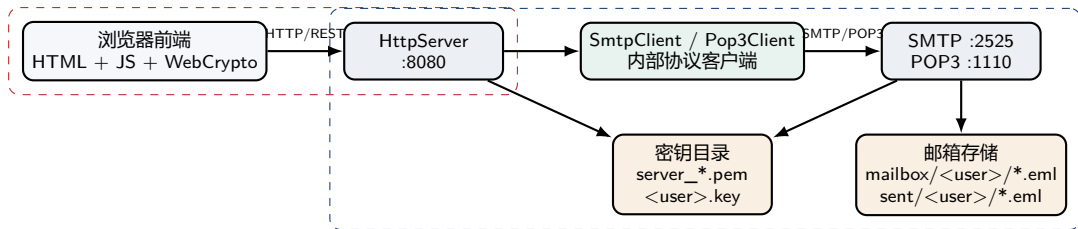
汇报口径

本项目把协议正确性和密码算法可实现性作为两条主线: 前者用 RFC 状态机加端到端测试证明, 后者用 NIST / REC 官方向量回归证明

总体架构：三服务器 + 两客户端 + 两加密层

层 1: RSA 数字信封

层 2: 自研对称加密



- 三个服务器各占一个线程，独立 accept 循环；每个连接再派发一个工作线程。
- HTTP 服务器扮演翻译官：浏览器只会 HTTP，后端用 SmtplibClient/Pop3Client 说 SMTP/POP3。

服务	协议	端口
SMTP 服务器	RFC 5321	2525
POP3 服务器	RFC 1939	1110
HTTP 服务器	Web/REST	8080

- 不用 25/110：低端口需要 root，且常被防火墙拦截；2525/1110 便于开发演示。
- 默认账号：alice / 123456、bob / 123456，保存在 users.txt。
- 注册接口 /api/register 写文件后即时生效——POP3 查询账号时会自动重读 users.txt。

一次完整收发路径

浏览器填写表单

- HTTP /api/send
- 可选层 1 RSA 信封解包
- 层 2 AES/ChaCha 加密载荷
- SMTP sendRawMail
- 服务器 saveMail 落盘
- POP3 RETR 取回
- 层 2 解密 / 附件解析
- 可选层 1 信封返回浏览器

```
MailForge/  
|-- Makefile  
|-- README.md / requirements.md  
|-- MailServer/  
| |-- main.cpp  
| |-- include/ src/  
| | |-- Server.* 网络基类  
| | |-- SmtplibServer.* SMTP 服务端  
| | |-- Pop3Server.* POP3 服务端  
| | |-- SmtplibClient.* SMTP 客户端  
| | |-- Pop3Client.* POP3 客户端  
| | |-- HttpServer.* Web 后端  
| | +-- MailCrypto.* 加密入口  
| |-- web/ 前端页面  
| |-- client_test.cpp 协议演示客户端  
| +-- users.txt  
-- crypto/  
| |-- aes.* chacha20.*  
| |-- rsa.* openssl_util.*  
| |-- random.*  
| +-- tests/ docs/  
+-- common/ base64 / logger / file_util / socket
```

分层思想

1. **公共层**：Socket 封装、Base64、日志、文件工具。
2. **协议层**：SMTP/POP3 服务端与客户端，纯文本状态机。
3. **Web 层**：HTTP 解析、REST 路由、会话、MIME。
4. **密码层**：MailCrypto 统一入口，按算法原语分为自研 / OpenSSL 两条线。
5. **展示层**：index.html 负责 Web 邮箱与 WebCrypto；demo_*.html 负责协议演示。

原理篇

协议状态机·网络模型·加密算法

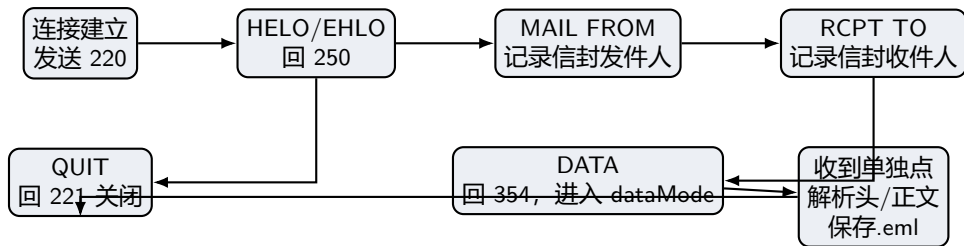
SMTP 原理：发信是一次先谈好再送内容的对话

- SMTP 是 **基于 TCP 的文本命令/响应协议**，默认服务器问候码 220。
- 核心流程：
 - EHLO/HELO：打招呼，服务器回 250；
 - MAIL FROM:<...>：声明信封发件人；
 - RCPT TO:<...>：声明收件人，可多个；
 - DATA：服务器回 354，客户端发送邮件原文；
 - 单独一行 . 表示结束，服务器回 250；
 - QUIT：退出，服务器回 221。
- 响应码：2xx 成功、3xx 中间状态、4xx 暂时失败、5xx 永久失败。
- 本项目只实现最核心命令，未实现认证、TLS、多收件人队列等扩展。

SMTP 对话示例

```
S: 220 MyMailServer ESMTP ready
C: EHLO client.example
S: 250 Hello client.example
C: MAIL FROM:<alice@example.com>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: Subject: Hello
C:
C: This is body.
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 Bye
```

SMTP 服务端状态机与实现要点



实现要点

- processCommand 用 dataMode 区分命令行和正文行。
- Smtplib 结构跨命令保存发件人、收件人、原文。
- parseMailData 解析头部区与正文区，提取 From/To/Subject/Date。

点填充 (Dot-Stuffing)

正文中以 . 开头的行，客户端发送时写成 . . ; 服务器收到后还原。否则正文里的单独一行 . 会被误认为 DATA 结束。

保存流程

1. 从 mail.to 取 @ 前部分，转小写；
2. 投递到 ./mailbox/<user>;
3. 文件名：时间戳 _ 随机数.eml；
4. 优先保留客户端头部；缺失的 Date/From/To/Subject 用信封信息补齐；
5. 头部区 + 空行 + 正文写入文件。

```
// SmtServer::saveMail
std::string user = mail.to;
if (at != npos) user = user.substr(0, at);
dir = "./mailbox/" + user;
mkdir(dir);
filename = ts + "_" + rand() + ".eml";
header = ...; // 补全标准头
file << header << "\r\n" << mail.text;
```

可改进点

rand() 非线程安全、文件名可能碰撞；应使用原子计数、随机数或 std::filesystem 唯一临时文件。

多用户隔离

bob@example.com -> mailbox/bob/; POP3 登录后只读自己的目录，天然隔离。

POP3 原理：下载式收件与三阶段状态机

- POP3 也是 TCP 文本协议，服务器先回 +OK，出错回 -ERR。
- 三阶段：
 - AUTHORIZATION**: USER / PASS, 认证;
 - TRANSACTION**: STAT / LIST / RETR / DELE / RSET / NOOP;
 - UPDATE**: QUIT 时真正删除被 DELE 标记的邮件。
- DELE 只打标记，符合 RFC 1939：中途断开不会误删。
- 多行响应单独一行 . 结束，正文里的 . 开头行同样要做点填充。
- 本项目还实现了 RSET 和 NOOP，并支持新注册账号即时登录。

POP3 对话示例

```
S: +OK MailForge POP3 ready
C: USER bob
S: +OK User bob known
C: PASS 123456
S: +OK mailbox ready, 2 message(s)
C: STAT
S: +OK 2 12345
C: LIST
S: +OK 2 message(s)
S: 1 6000
S: 2 6345
S: .
C: RETR 1
S: +OK 6000 octets
... mail content ...
S: .
C: DELE 1
S: +OK message 1 deleted
C: QUIT
S: +OK signing off
```

关键数据结构

- Pop3State: 认证标志、用户名、userKey、邮件快照 `std::vector<Pop3Mail>`。
- Pop3Mail: 文件路径、大小、deleted 标记。
- 登录成功时 `loadUserMails()` 扫描 .eml, 排序后生成稳定编号。

认证与账号热加载

- `normalizeUser()` 统一小写并去掉 @ 域名；
- `isUserKnown/checkPassword` 内存表没有则重读 `users.txt`；
- 登录成功顺便 `ensureUserKey()` 初始化对称密钥。

- `sendMailContent()` 用 `std::getline` 逐行读文件，处理 `\r` 和点填充，避免一次性载入大邮件。
- QUIT 时才 `unlink()` 被标记文件，实现 UPDATE 阶段。
- HTTP 后端调用
`Pop3Client::login/list/retr/dele/quit`，与外部邮件客户端走同一套协议。

可改进点

未实现 UIDL、TOP、APOP；无并发删除保护；认证明文传输。

通用网络基类：Socket 生命周期与多线程

```
1 class Server {
2 public:
3     Server(int port);
4     virtual ~Server();
5     bool start();
6     void stop();
7 protected:
8     virtual void handleClient(int fd) = 0;
9 private:
10    int server_fd;
11    int port;
12    std::atomic<bool> is_running;
13 };
```

start() 固定流程

socket() -> setsockopt(SO_REUSEADDR)
-> bind() -> listen(5) -> accept() 循环

并发模型

每接受一个连接，创建 `std::thread` 并 `detach()`；
子类只需实现 `handleClient`，
SMTP/POP3/HTTP 复用同一套 `accept` 逻辑。

可改进点

- 一连接一线程，高并发下线程开销大；应使用线程池 / `epoll`。
- `detach()` 线程 + `stop()` 关闭 `fd` 存在竞态；缺少优雅退出。
- 仅支持 IPv4；未设置 `SO_REUSEADDR` 以外的连接选项。

HTTP 解析

- `readLine()` 逐字节读到 `\textbackslash n`, 兼容 `\r \n` ;
- `readRequest()` 解析请求行、查询字符串、头部、Content-Length body;
- POST 表单按 `a=b\&c=d` 解析并做 URL 解码;
- 响应固定 Connection: close, 循环 send 保证发完。

会话与认证

`/api/login` 内部用 `Pop3Client` 试登录, 成功则生成 token 存入 `sessions_`; 后续接口用 token 查用户。

```
1 void HttpServer::handleClient(int fd) {
2     HttpRequest req; HttpResponse resp;
3     if (!readRequest(fd, req)) return;
4     route(req, resp);
5     sendHttp(fd, resp);
6 }
7 void HttpServer::route(req, resp) {
8     if (req.path.rfind("/api/",0)==0)
9         handleApi(req, resp);
10    else
11        handleStatic(req, resp);
12 }
```

可改进点

未实现长连接、分块传输、HTTP 方法限制、请求体大小上限; 静态文件路径未做严格防目录穿越。

方法	路径	功能
POST	/api/register	注册新用户, 写入 users.txt
POST	/api/login	通过 POP3 验证账号, 返回 token
POST	/api/logout	删除会话
POST	/api/send	发信; 支持 encrypt=1\&algo=aes chacha; 支持 env RSA 信封
GET	/api/inbox	POP3 LIST + RETR, 返回收件箱 JSON
GET	/api/mail?n=	读取指定邮件, 自动解密, 解析附件列表
GET	/api/attachment?n=\&i=	下载附件, web=1 时走 RSA 信封
POST	/api/delete	POP3 DELE + QUIT 删除邮件
GET	/api/sent	本地 sent/<user>/ 已发送列表
GET	/api/sent/mail?n=	读取已发送邮件 (加密副本用发件人密钥解密)
GET	/api/benchmark	100 封约 1MB 邮件压测: plain/aes/chacha/all
GET	/api/webkey	下发服务器 RSA 公钥 (层 1)
POST	/api/webpub	上传浏览器 RSA 公钥 (层 1 响应封装)

邮件格式与附件: MIME multipart + Base64

生成附件邮件

1. 浏览器上传文件名和 Base64 内容;
2. 生成随机 boundary;
3. 组装 multipart/mixed: 正文段 + 附件段;
4. 附件段使用 Content-Transfer-Encoding: base64;
5. 与加密叠加时, 整个 multipart 作为一个字符串先加密, 再放进邮件正文。

解析附件

- 从头部找 boundary;
- 按 boundary 切分 MIME 段;
- 解析 Content-Disposition: attachment、filename、Content-Type;
- 下载时 Base64 解码;
- 附件名做简单消毒, 防止响应头注入。

可改进点

未做严格的 MIME 递归解析、编码头 (RFC 2047) 解码; 附件名中文可进一步提升。

两层加密模型：为什么需要两层？



层 1：保护浏览器到服务器

- 解决本机没有 TLS 时 Web 表单、收件箱、附件在 HTTP 明文传输的问题。
- 用标准算法：AES-256-GCM 做内容加密，RSA-OAEP-2048 包装会话密钥。
- 浏览器端 WebCrypto，服务器端 OpenSSL EVP。

层 2：保护服务器内部与磁盘

- 邮件正文/附件整体加密后再走 SMTP、POP3 和落盘。
- 算法可切换：AES-256-CBC（分组）或 ChaCha20（流密码）。
- 纯 C++ 自研，不依赖 OpenSSL，体现密码算法课程实践。
- 发送与已发送副本分别用收件人/发件人密钥加密。

层 1: RSA 数字信封原理

发送方向：浏览器到服务器

1. 浏览器生成一次性 **AES-256-GCM 会话密钥**;
2. 用服务器 RSA 公钥做 **RSA-OAEP(SHA-256)**, 封装 32 字节会话密钥;
3. 表单明文 (to/subject/body/附件) 用 AES-GCM 加密, 末尾带 16 字节认证 tag;
4. 拼成 `k=...|iv=...|ct=...` 放入 `env` 字段 POST `/api/send`;
5. 服务器用 RSA 私钥拆封, 再用会话密钥解 GCM。

为什么是混合加密?

- RSA 慢且单次加密长度有限 (2048 位 OAEP-SHA256 约 190 字节);
- AES-GCM 快, 可加密任意长表单, 且提供完整性认证;
- RSA 只负责安全地把会话密钥交给对方。

反向：服务器到浏览器

浏览器生成 RSA-2048 密钥对, 公钥上传 `/api/webpub`; 服务器读信、附件、压测结果用该公钥封装信封返回, 浏览器私钥解封。

层 1 互通细节: WebCrypto 与 OpenSSL 如何对上

服务器端 OpenSSL

- RSA 密钥: server_public.pem / server_private.pem, 首次启动自动生成;
- webEnvelopeOpen(): 先 Base64 解码 k=, Rsa::PrivateDecrypt 还原会话密钥;
- AesGcmDecrypt() 校验 GCM tag;
- webEnvelopeSeal() 反向操作。

```
// 信封线格式  
k=<Base64 RSA-OAEP-SHA256(32B session key)>  
|iv=<Base64 12B GCM IV>  
|ct=<Base64 AES-GCM(ciphertext || 16B tag)>
```

- 浏览器: crypto.subtle.importKey("spki", ...) 导入服务器公钥;
- JWK 私钥存 localStorage, 多次刷新仍可解封历史响应;
- 若 WebCrypto 不可用, 前端降级为明文 HTTP, 并在控制台告警。

层 2：自研对称加密的两种算法

AES-256-CBC

- 分组密码，块 16 字节，密钥 32 字节，14 轮；
- 工作模式：CBC，每个明文块先与前一块密文异或；
- 填充：PKCS#7，解密后校验填充；
- 每封邮件随机 16 字节 IV；
- 线格式：MailForge::ENC::AES:: + Base64(IV || 密文)。

ChaCha20

- 流密码，密钥 32 字节，nonce 12 字节，64 字节密钥流块；
- ARX 结构：加法、异或、循环移位，无查表；
- 20 轮（10 次列轮 + 对角轮）；
- 每封邮件随机 12 字节 nonce，从计数器 0 开始；
- 线格式：MailForge::ENC::CHA:: + Base64(nonce || 密文)。

关键工程约定

加密端把真实 Subject + 空行 + 正文/附件 multipart 整体加密；邮件头部只保留 Subject：[加密邮件] 和 X-MailForge-Crypto 标记。接收端按魔数头自动识别算法并用当前用户密钥解密。

AES-256 原理：有限域上的四个变换

密钥扩展

32 字节密钥扩展为 60 个 32 位字 (15 个轮密钥)。
AES-256 在 $i \% 8 == 4$ 时多一次 SubWord。

轮函数

1. **SubBytes**: 每个字节通过 S 盒做非线性替换;
2. **ShiftRows**: 状态矩阵按行循环左移;
3. **MixColumns**: 每列在 $GF(2^8)$ 上乘固定矩阵;
4. **AddRoundKey**: 与轮密钥异或。

最后一轮不做 MixColumns; 解密按逆变换反序执行。

S 盒不是抄表

- 在 $GF(2^8)$ 中求乘法逆元: $a^{-1} = a^{254}$;
- 再做仿射变换 $b \mapsto Ab \oplus 0x63$;
- 逆 S 盒由正向映射反推生成。

CBC + PKCS#7

- $C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = IV$;
- 解密 $P_i = D_K(C_i) \oplus C_{i-1}$;
- 明文补 pad 个字节, 值为 pad ; 解密后校验。

AES-256-CBC 代码实现要点

```
1 void ExpandKey(const unsigned char key[32],
2               unsigned char rk[240]);
3 void EncryptBlock(const unsigned char rk[240],
4                  const unsigned char in[16],
5                  unsigned char out[16]);
6 void DecryptBlock(...);
7 bool Aes::Encrypt(key, iv, plaintext, ciphertext) {
8     size_t pad = 16 - (plaintext.size() % 16);
9     // PKCS#7 填充
10    ExpandKey(key.data(), rk.data());
11    for each block:
12        blk = plaintext_block ^ prev;
13        EncryptBlock(rk, blk, out);
14        prev = out;
15 }
```

正确性验证

crypto/tests/test_crypto.cpp 中使用 **NIST SP 800-38A F.2.3** 官方向量回归 AES-256-CBC。

安全边界

CBC 本身只提供机密性，不提供完整性认证。当前主要靠 PKCS#7 填充校验顺便发现错误；更严谨应使用 AES-GCM 或 Encrypt-then-MAC。

状态矩阵 (16 个 32 位字)

$$\begin{pmatrix} c_0 & c_1 & c_2 & c_3 \\ k_0 & k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 & k_7 \\ t_0 & n_0 & n_1 & n_2 \end{pmatrix}$$

4 个常量、8 个密钥字、1 个计数器、3 个 nonce 字。

Quarter Round

$a += b;$	$d \oplus = a;$	d	$\lll 16$
$c += d;$	$b \oplus = c;$	b	$\lll 12$
$a += b;$	$d \oplus = a;$	d	$\lll 8$
$c += d;$	$b \oplus = c;$	b	$\lll 7$

20 轮 = 10 次双轮

- 列轮: $(0, 4, 8, 12), (1, 5, 9, 13), (2, 6, 10, 14), (3, 7, 11, 15);$
- 对角轮: $(0, 5, 10, 15), (1, 6, 11, 12), (2, 7, 8, 13), (3, 4, 9, 14);$
- 每轮后累加初始状态, 小端输出 64 字节密钥流;
- 密文 = 明文 $\oplus$ 密钥流。

验证

使用 [RFC 8439 §2.3.2](#) 官方向量验证 Block() 输出。


```
1 void ChaCha20::Block(key, nonce, counter, out) {
2     uint32_t state[16];
3     state[0..3] = constants;
4     state[4..11] = key words (LE);
5     state[12] = counter;
6     state[13..15] = nonce words (LE);
7     uint32_t work[16] = state;
8     for (int round = 0; round < 10; ++round) {
9         QuarterRound(work, 0, 4, 8, 12);
10        /* ... column rounds ... */
11        QuarterRound(work, 0, 5, 10, 15);
12        /* ... diagonal rounds ... */
13    }
14    for (int i = 0; i < 16; ++i)
15        Store32LE(work[i] + state[i], out + 4*i);
16 }
```

加解密统一接口

Crypt() 按 64 字节分块，逐块生成密钥流，密文与明文长度相同。

安全边界

- 流密码不提供完整性认证，当前用解密结果是否以 Subject: 开头做启发式兜底；
- 生产环境应使用 ChaCha20-Poly1305，或 Encrypt-then-MAC；
- 同一密钥下 nonce 绝不能重复。

对称密钥管理与加解密挂钩点

密钥文件

- 每个账号 `keys/<user>.key`: 32 字节随机密钥;
- 首次使用 `getUserKey()` 自动生成;
- 用户名规范化: 邮箱取 @ 前、转小写、过滤非法字符;
- 全局互斥锁 `gKeyMutex` 保护查文件加写文件。

服务端 RSA 密钥

`keys/server_public.pem` / `server_private.pem`, 首次启动自动生成; 用于层 1。

加解密挂钩点

1. `handleSend()`: 组装载荷后用收件人密钥加密;
2. `decodeMail()`: 按魔数识别 AES/CHA, 用查看者密钥解密;
3. 已发送副本: 用发件人自己的密钥再加密一份;
4. 附件下载: 先解邮件, 再解析 MIME, 再 Base64 解码。

可改进点

密钥文件明文存盘、无 KDF、无权限控制; 应该用 OS 密钥环、主密钥派生、文件权限 0600。

密码算法正确性：用官方向量说话

算法	验证来源	测试内容
Base64	RFC 4648	编码 / 解码往返、换行兼容
AES-256-CBC	NIST SP 800-38A F.2.3	官方向量逐块比对
ChaCha20	RFC 8439 §2.3.2	密钥流块官方向量比对
RSA 信封	OpenSSL + WebCrypto	Seal/Open 双向互通、篡改/错密钥拒绝
协议	端到端测试	SMTP 发信 -> 落盘 -> POP3 收信
性能	100 封约 1MB	成功率、丢包率、平均时延、解密还原率

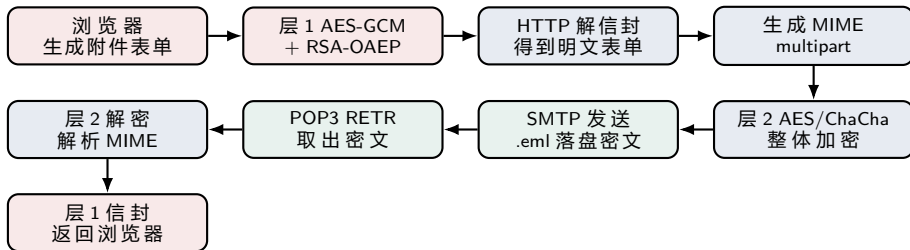
测试入口

```
make test-crypto: base64 / logger / socket /  
crypto  
make test-rsa: RSA 数字信封  
网页 /api/benchmark: 三种模式全跑
```

测试改进方向

增加协议模糊测试、异常报文测试、并发测试、跨平台 CI；把压测从本地同账号扩展到不同账号和多客户端。

端到端数据流：一封加密附件邮件的完整旅程



- 明文只在浏览器、HTTP 进程内、POP3 客户端解密后短暂存在；SMTP/POP3 协议链路与磁盘看到的是层 2 密文。
- 层 1 保护浏览器到 HTTP 进程；已发送副本用发件人密钥再次加密，解决发件人回看问题。

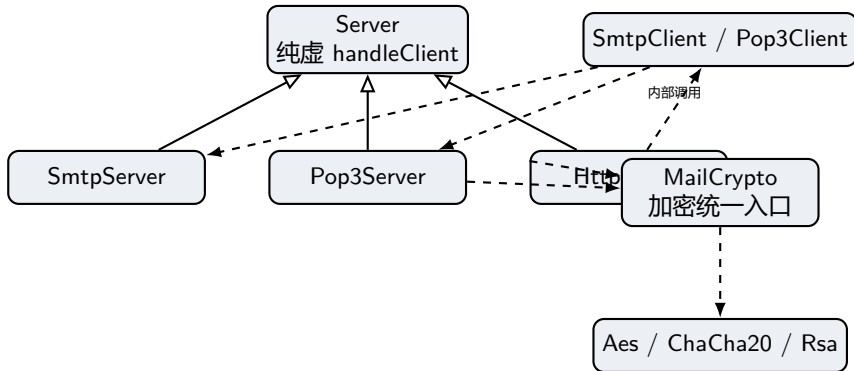
结构篇

模块划分·类关系·数据流·构建测试

模块职责划分

模块	文件	职责
网络基类	Server.h/cpp	socket/bind/listen/accept, 一连接一线程, 纯虚 handleClient
SMTP 服务端	SmtpServer.*	命令状态机、DATA 点填充、头部解析、按用户落盘
POP3 服务端	Pop3Server.*	三阶段状态机、账号加载、邮件快照、DELE/QUIT 删除
SMTP 客户端	SmtpClient.*	内部发信, 支持 sendRawMail 发送已加密原文
POP3 客户端	Pop3Client.*	内部收信, 支持 login/stat/list/retr/dele/quit
HTTP/Web	HttpServer.*	HTTP 解析、REST 路由、会话、JSON、MIME、压测
邮件加密	MailCrypto.*	自研 AES/ChaCha 入口、密钥管理、RSA 信封
密码原语	crypto/*	AES、ChaCha20、RSA、CSPRNG、OpenSSL 工具
公共工具	common/*	Base64、logger、file_util、socket 封装
前端	web/*.html	Web 邮箱、WebCrypto 信封、SMTP/POP3

核心类关系



- **模板方法模式**：Server::start() 固定网络流程，子类只实现 handleClient()。
- **协议与加密解耦**：SMTP/POP3 服务器不关心正文是明文还是密文；加密在 HTTP 发送前和读取后挂钩。
- **统一加密入口**：MailCrypto 对上层暴露 encryptPayload/decryptPayload/getUserKey，隐藏算法细节。

```
1 struct SmtMail {
2     std::string from, to, body;
3     std::string headers, text;
4     std::string headerFrom, headerTo;
5     std::string subject, headerDate;
6 };
7
8 struct Pop3Mail {
9     std::string path;
10    long long size;
11    bool deleted;
12 };
13
14 struct Pop3State {
15     bool authed = false;
16     std::string username, userKey;
17     std::vector<Pop3Mail> mails;
18 };
```

```
1 struct HttpRequest {
2     std::string method, path, version;
3     std::map<std::string, std::string> query, form;
4 };
5
6 struct HttpResponse {
7     int status;
8     std::string contentType;
9     std::string extraHeaders;
10    std::string body;
11 };
12
13 struct Session {
14     std::string user, pass;
15     std::string webPubPem;
16 };
```

设计观察

结构体轻量、可直接拷贝；但 Session.pass 明文保存、缺少过期时间，是安全改进点。

磁盘布局

```
MailServer/
|-- mailbox/<user>/*.eml 收件箱
|-- sent/<user>/*.eml 已发送
|-- users.txt 用户名:密码
+-- keys/
    |-- <user>.key 32B 对称密钥
    |-- server_public.pem
    +-- server_private.pem
```

会话管理

- token 由时间戳、rand()、PID 拼接;
- sessions_ 受互斥锁保护;
- 登录成功后可选登记浏览器 RSA 公钥;
- 没有 token 过期与主动清理。

加密数据落盘

加密邮件的 .eml 正文是

MailForge::ENC::AES/CHA:: 密文; 主题为占位符,
附件随 multipart 整体加密。

```
make server # 编译 MailServer
make run # 启动 SMTP/POP3/HTTP
make client # 编译协议演示客户端
make test-crypto # Base64/AES/ChaCha 官方向量
make test-rsa # RSA 信封测试
make clean # 清理产物
```

环境要求

Linux/WSL2, g++ C++17, make, OpenSSL 开发库 (libssl-dev)。层 2 自研加密测试不依赖 OpenSSL。

演示入口

<http://localhost:8080/> 主界面
[/demo_smtp.html](#) SMTP 手敲终端
[/demo_pop3.html](#) POP3 手敲终端

创新点 两层加密·纯自研密码·浏览器互通·工程闭环

密码学创新

1. **两层加密模型**: Web 链路与服务器内部/落盘分别保护。
2. **纯 C++ 自研 AES-256-CBC 与 ChaCha20**: 不调 OpenSSL, 过 NIST/RFC 官方向量。
3. **WebCrypto 与 OpenSSL 互通**: 浏览器原生 API 与 C++ 服务端互解数字信封。
4. **加密与协议解耦**: SMTP/POP3 服务器完全不感知密文内容。

系统工程创新

5. **协议、Web、存储统一.eml**: HTTP 后端走本机 SMTP/POP3, 不绕开协议。
6. **内建协议演示终端与性能压测**: 浏览器手敲协议命令, 一键 100 封压测并自动清理。
7. **零配置密钥管理**: 用户密钥、服务器 RSA 密钥首次使用自动生成。
8. **发件人副本再加密**: 加密邮件用发件人密钥另存一份, 收件箱和发件箱都能读。

创新点 1-2：两层加密与纯自研算法

创新 1：两层加密模型

- 层 1: AES-256-GCM + RSA-OAEP, 保护浏览器到服务器;
- 层 2: AES-256-CBC / ChaCha20, 保护服务器端口之间与磁盘;
- 两层的信任边界不同, 组合起来覆盖 HTTP 明文和内部链路/存储。

创新 2：自研算法可验证

- S 盒由 $GF(2^8)$ 求逆 + 仿射变换生成;
- CBC、PKCS#7、密钥扩展全部手写;
- ChaCha20 按 RFC 8439 实现 ARX 和 20 轮;
- 官方测试向量回归, 避免自己加密自己解密的假验证。

答辩要点

自研不是拒绝标准库, 而是为了课程目标展示密码算法内部结构; 生产环境仍应优先使用经过审计的密码库和 AEAD。

可量化结果

make test-crypto 全通过;
make test-rsa 全通过;
100 封约 1MB 邮件加密发送与解密还原 100/100。

创新点 3-4：浏览器密码学互通与协议透明

创新 3：WebCrypto 与 OpenSSL 互通

- 浏览器用原生 `crypto.subtle`，不引入 JS 第三方库；
- 服务器用 OpenSSL EVP；
- 双方约定标准线格式，PEM/SPKI、RSA-OAEP-SHA256、AES-GCM 全部对齐；
- 浏览器维持 RSA 密钥对，服务器可以反向封装收件箱、附件、压测结果。

创新 4：加密与协议解耦

- `SmtpClient::sendRawMail()` 只负责发原文，不关心是否加密；
- SMTP/POP3 服务器只看到密文正文，仍按标准协议处理；
- 外部 Python `smtpplib/poplib` 也能连接，协议兼容性不被破坏。

意义

加密不是散落在各协议里的 if-else，而是统一挂在 HTTP 收发边界，层次清晰、易替换算法。

创新点 5-6：工程闭环与演示能力

创新 5：统一.eml 存储

- SMTP 投递、POP3 读取、Web 收发都围绕同一份 .eml；
- 邮件按用户目录隔离，重启不丢；
- 已发送副本独立保存，支持列表、阅读、附件下载、删除。

创新 6：内建演示与压测

- demo_smtp.html / demo_pop3.html：浏览器里像 telnet 一样手敲命令；
- /api/benchmark：plain/aes/chacha/all 一键 100 封；
- 统计 SMTP 成功、收件数、丢包率、平均时延、解密还原数，测完自动清理。

其他加分项

- 零配置密钥：首次使用自动生成；
- 发件箱加密副本：发件人自己可读；
- 编码兼容：中文主题/正文、Base64 附件、点填充；
- 结构化日志和 ASCII 表格便于现场讲解。

测试结果：功能、加密、性能

功能端到端

- SMTP 发信 -> mailbox/<user>/*.eml -> POP3 收信；
- 中文主题、正文、附件、点填充、删除语义、多用户隔离；
- Web 注册/登录/收发/附件/已发送闭环。

加密端到端

- 加密发送后落盘正文只有 MailForge::ENC::AES/CHA:: 密文；
- 收件人读取自动解密还原；
- 明文通道不受影响。

性能压测

- 每模式连发 100 封约 1MB 邮件；
- 发送 100/100，丢包率 0%；
- 加密邮件逐封解密还原 100/100；
- AES 平均单封约 100ms，ChaCha20 约 60ms，远低于 2s 指标。

注意

以上为本地环回环境的课程演示数据；真实网络需要更严格的统计与外部 SMTP 测试。

现场演示脚本：8 分钟走完全部亮点

1. **启动**：make run，浏览器打开 localhost:8080。
2. **注册/登录**：用 bob 登录，说明 Web 登录复用 POP3 认证。
3. **明文发信**：给 alice 发一封带附件邮件，展示收件箱与附件下载。
4. **加密发信**：勾选发送时加密，分别选 AES 与 ChaCha。
5. **落盘验证**：终端 cat mailbox/alice/*.eml，展示密文魔数头。
6. **解密收信**：alice 登录，阅读自动解密，展示真实主题和附件。
7. **协议终端**：打开 demo_smtp.html 手敲 EHLO/MAIL/RCPT/DATA。
8. **压测**：选择三种全跑，等待结果表，展示成功率与平均时延。
9. **Web RSA**：勾选层 1，展示请求/响应信封与白

讲述节奏建议

内容	建议时长
背景与目标	1 min
架构与协议原理	3 min
加密原理	4 min
代码结构	2 min
创新点	2 min
演示	4 min
改进与总结	2 min

时间控制

若总时长 15 分钟，原理和加密各压缩 1 分钟，演示只保留明文发信、加密落盘、压测三项。

还可以修改的地方 安全·架构·功能·测试

改进点总览：优先级建议

类别	问题	风险	优先级
安全	明文密码、无 TLS、无认证加密	高	P0
安全	RSA 信封无签名、无重放保护	高	P0
架构	一连接一线程、detach、无优雅退出	中高	P1
架构	邮件全量载入内存、无流式加解密	中	P1
功能	无 STARTTLS、SMTP AUTH、多收件人	中	P1
功能	无搜索、分页、回复、转发、草稿	低	P2
工程	无配置、无数据库、无日志轮转	中	P1
测试	协议自动化/模糊/并发测试不足	中高	P1

答辩建议

先讲安全类 P0，再讲架构和测试；不要把改进点讲成项目失败，而是讲清楚当前版本的设计权衡与下一步路线。

认证与传输安全

- 密码明文存 `users.txt` 与 `Session.pass`：应使用 Argon2/bcrypt/scrypt 加盐哈希，会话只存用户 ID。
- SMTP/POP3/HTTP 均无 TLS：应实现 STARTTLS、POP3S/HTTPS，至少把层 1 信封升级为真正 TLS。
- Web 登录无失败限速、无验证码、无令牌过期：应加速率限制、会话 TTL、登录审计。
- `randomToken()` 用 `rand()`：应改用 CSPRNG 生成 128 位以上 token。

密码协议

- AES-CBC 与 ChaCha20 都无认证：应换成 AES-GCM 或 ChaCha20-Poly1305，或 Encrypt-then-MAC。
- 层 1 RSA 信封没有数字签名：服务器返回内容可被中间人替换；应加签名/证书。
- 无防重放：同一信封可重复提交；应加 nonce、时间戳、请求 ID。
- 密钥裸存：应使用 KDF/主密钥、OS keyring、文件权限 0600。
- 服务器 RSA 私钥明文 PEM：应加密存储并使用密钥管理服务。

架构与工程：从课程版走向可维护版本

并发与资源

- 一连接一线程 + detach：高并发下资源不可控，且 stop() 与工作线程存在竞态。
- 建议：epoll/kqueue 或线程池；std::jthread/RAII 管理生命周期；增加优雅停机。
- sendAll/recvLine 等逻辑在 SMTP/POP3/HTTP 中重复，可提取公共 Connection 类。

存储与性能

- 邮件和附件全量读入内存，未做流式加解密；大附件/并发时内存压力大。
- 建议：分块流式 AES-GCM/ChaCha20，临时文件 + 原子 rename，限制单封大小。
- rand() 文件名/边界可能碰撞，应使用 CSPRNG

配置与部署

- 端口、路径、域名、算法硬编码：应引入配置文件/环境变量。
- 无数据库：users.txt 和目录扫描在用户量增大后性能与并发都不佳；可迁移 SQLite/PostgreSQL。
- 无日志分级、轮转、审计：应统一 logger，记录登录、发信、解密失败等安全事件。
- Makefile 面向 Linux/POSIX；Windows 原生编译需要 unistd/dirent 等兼容层。

代码质量

增加静态分析、格式化、单元测试覆盖率、CI；处理编译警告，补文档和接口注释。

功能与协议：补齐邮件系统常用能力

协议层

- SMTP: STARTTLS、AUTH、多收件人、退回队列、重试策略、邮件大小限制。
- POP3: UIDL、TOP、APOP/STLS, 删除并发保护与跨会话一致性。
- 可增加 IMAP: 支持服务器端文件夹、已读/未读状态、同步多设备。
- 真正连公网邮箱需要 DNS MX 查询、出站队列、SPF/DKIM/DMARC。

Web 邮箱功能

- 搜索、分页、邮件线程、回复/转发、草稿箱、通讯录。
- HTML 邮件与安全的富文本渲染；附件预览、多附件、拖拽上传。
- 邮件状态（已读/未读/标星）、文件夹、批量操作。
- 前端体验：发送进度、错误重试、移动端适配、国际化。

格式兼容

MIME 递归解析、RFC 2047 编码头、HTML/纯文本 alternative、正确的 UTF-8 头部编码。

测试与评测：让结论更可信

测试覆盖

- 协议异常测试：错误状态码、超长行、半包/粘包、非法命令、点填充边界。
- 模糊测试：SMTP/POP3/HTTP 报文随机变异，验证不崩溃、不越界。
- 并发测试：多用户同时收发、同时删除、同时读写同一账号。
- 加密测试：错误密钥、篡改密文、nonce 复用检测、填充 oracle 场景。

性能评测

- 当前压测是本地环回、同一账号、同一服务器，仅适合功能演示。
- 应区分本机 vs 局域网 vs 公网，不同附件大小，不同并发数，统计 P50/P95/P99。
- 测时延应包含 DNS/TCP/TLS/SMTP/POP3 全链路，而非只统计 SMTP 发送。
- 成功率应按首次成功、最终成功分别统计，并保留失败原因。

工程化

接入 GitHub Actions / GitLab CI，执行编译、单测、静态检查；生成覆盖率报告和基准对比。

我们完成了什么

- 从零手写 SMTP、POP3、HTTP 服务器和协议客户端；
- 实现多用户 Web 邮箱、MIME 附件、.eml 持久化；
- 实现两层加密：Web RSA 信封 + 服务器内部/落盘自研 AES/ChaCha；
- 用 NIST/RFC 官方向量、端到端测试和 100 封压测验证；
- 提供协议演示终端和可视化压测结果。

核心收获

- 协议状态机与 TCP 流式处理必须考虑粘包/半包、点填充、超时；
- 密码算法不能只写通，必须用官方向量证明正确性；
- 加密要按信任边界分层，协议层与密码层解耦；
- 工程化和安全加固和功能实现同样重要。

一句话

MailForge 是一个可运行、可演示、可验证的邮件系统课程设计，也是一套继续迭代到生产级的安全与工程蓝图。

答辩问答准备：高频问题与回答要点

协议类

- 如何处理 TCP 粘包/半包？答：buf 按 \n 拆行，半行留在缓冲区。
- POP3 删除什么时候生效？答：DELE 只标记，QUIT 进入 UPDATE 才 unlink。
- DATA 里的点开头怎么办？答：客户端点填充，服务器还原。
- 邮件头与信封地址是什么关系？答：SMTP 路由用信封，展示用头部，服务器会补齐缺失头部。

密码类

- 为什么两层？答：信任边界不同：浏览器到服务器与服务器内部/磁盘。
- 为什么自研 AES/ChaCha？答：课程目标是理解算法；用官方向量保证正确。
- 自研算法安全吗？答：教学验证用；生产应使用审计过的 AEAD 库。
- RSA 信封为什么还要 AES？答：RSA 慢且长度受限，只用于包装会话密钥。
- 中间人能否篡改？答：GCM tag 防篡改；但信封无签名与证书，仍有中间人风险，这是改进点。

谢谢！ 请各位老师批评指正 MailForge：让邮件协议和
安全算法都看得见、跑得通、验得过